

High Availability GCP Web Application

Status: Approved by reviewer
Run: 8efd226b - 2026-10-08 12:51
Domains: fullstack, cloud

1. Executive summary

Design a highly available web application architecture on Google Cloud Platform (GCP) capable of supporting 100,000 users, including a robust disaster recovery strategy.

Project type: High Availability Multi-Region GCP Web Application - large complexity. Requires complex multi-region Google Cloud infrastructure, GKE orchestration, global load balancing, and cross-region database replication to support 100,000 users.

At a glance: 7.5 FTE team, about 135 person-days of effort, about 14 weeks to deliver.

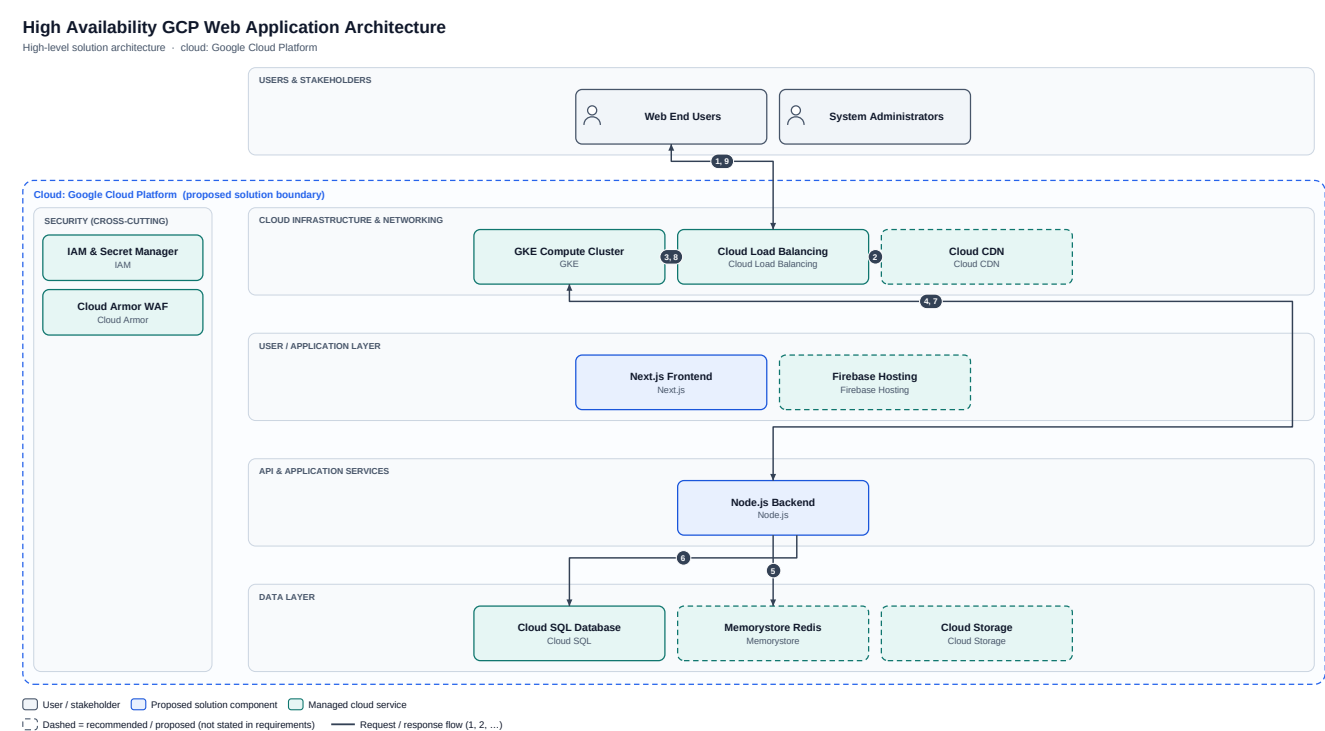
Reviewer: The combined solution thoroughly addresses all requirements for a highly available, multi-region GCP web application supporting 100,000 users. The architecture incorporates robust disaster recovery strategies, global load balancing, auto-scaling compute via GKE, managed databases with multi-region replication, and comprehensive security controls. No critical or high findings were identified.

2. Solution architecture

This solution delivers a robust, globally available web application architecture on Google Cloud Platform designed to effortlessly support 100,000 concurrent users with automated disaster recovery and low latency.

2.1 High-level solution architecture diagram

High-level solution architecture



Cloud: Google Cloud Platform. Boxes are grouped by layer and coloured by type (client system, proposed component, third-party, managed cloud service, AI, data store). Dashed boxes are recommended additions that the requirement did not state.

2.2 End-to-end data flow

Web Application Request Flow

- 1. Web End Users → Cloud Load Balancing: HTTPS request
- 2. Cloud Load Balancing → Cloud CDN: Check edge cache
- 3. Cloud Load Balancing → GKE Compute Cluster: Route to GKE
- 4. GKE Compute Cluster → Node.js Backend: Process business logic
- 5. Node.js Backend → Memorystore Redis: Fetch session cache
- 6. Node.js Backend → Cloud SQL Database: Query relational data
- 7. Node.js Backend → GKE Compute Cluster: Return API response
- 8. GKE Compute Cluster → Cloud Load Balancing: Forward response
- 9. Cloud Load Balancing → Web End Users: HTTP response

2.3 Major components

Layer	Component	Technology	Type	Status	Purpose
Users & stakeholders	Web End Users	-	actor	required	Represents the 100,000 active users accessing the web application globally.
Users & stakeholders	System Administrators	-	actor	required	Manages platform operations, infrastructure deployments, and security oversight.
User / application layer	Next.js Frontend	Next.js	proposed	required	Provides high-performance server-side rendering and responsive user interfaces.
User / application layer	Firebase Hosting	Firebase Hosting	cloud service	recommended	Delivers static assets and frontend code globally via a fast CDN edge infrastructure.
API & application services	Node.js Backend	Node.js	proposed	required	Executes core business logic and REST API operations to serve user requests.
Data layer	Cloud SQL Database	Cloud SQL	cloud service	required	Persists transactional relational data with cross-region replication for disaster recovery.
Data layer	Memorystore Redis	Memorystore	cloud service	recommended	Stores user sessions and caches frequent queries for low-latency responses.
Data layer	Cloud Storage	Cloud Storage	cloud service	recommended	Stores durable user media files and system backups.
Cloud infrastructure & networking	GKE Compute Cluster	GKE	cloud service	required	Orchestrates containerised backend services with automated horizontal scaling across regions.
Cloud infrastructure & networking	Cloud Load Balancing	Cloud Load Balancing	cloud service	required	Distributes incoming traffic across multi-region backends with automated failover.
Cloud infrastructure & networking	Cloud CDN	Cloud CDN	cloud service	recommended	Caches static content at edge locations to minimize user latency and origin load.
Security	IAM & Secret Manager	IAM	cloud service	required	Enforces least privilege access and securely manages application secrets.
Security	Cloud Armor WAF	Cloud Armor	cloud service	required	Protects the web application against DDoS attacks and layer 7 vulnerabilities.

2.4 Requirement traceability

Requirement	Delivered by
Serve web traffic efficiently to 100,000 users	Cloud Load Balancing, Cloud CDN, GKE Compute Cluster, Node.js Backend
Failover mechanism in case of a regional outage	Cloud Load Balancing, GKE Compute Cluster, Cloud SQL Database
High availability (e.g., 99.99% uptime)	Cloud Load Balancing, GKE Compute Cluster, Cloud SQL Database, Memorystore Redis
Low latency for global or regional users	Cloud CDN, Cloud Load Balancing, Firebase Hosting
RTO (Recovery Time Objective) and RPO (Recovery Point Objective) definition for disaster recovery	Cloud SQL Database, GKE Compute Cluster, Cloud Load Balancing

2.5 Key assumptions

- Standard web application architecture with a database and backend services is sufficient.

- Multi-region or multi-zone GCP deployment is required for high availability and disaster recovery.
- Application containers are stateless and ready for horizontal auto-scaling on GKE.

2.6 Recommended technology stack

Layer	Technology	Purpose	Status
User / application layer	Next.js	Frontend framework supporting server-side rendering and global performance.	required
User / application layer	Firebase Hosting	Global static content hosting and edge distribution.	recommended
API & application services	Node.js	Asynchronous backend runtime handling concurrent user API requests.	required
Data layer	Cloud SQL	Managed relational database with cross-region replication for disaster recovery.	required
Data layer	Memorystore	In-memory Redis cache for fast session management and query acceleration.	recommended
Data layer	Cloud Storage	Durable object storage for user media and system backups.	recommended
Cloud infrastructure & networking	GKE	Container orchestration and automated horizontal pod autoscaling.	required
Cloud infrastructure & networking	Cloud Load Balancing	Global anycast HTTP(S) load balancing and multi-region failover.	required
Cloud infrastructure & networking	Cloud CDN	Global edge caching for static assets and reduced origin load.	recommended
Security	IAM	Least privilege access control across all Google Cloud resources.	required
Security	Cloud Armor	DDoS protection and Web Application Firewall security policies.	required

2.7 Security considerations

- Enforce TLS 1.3 encryption for all traffic in transit across the load balancers and services.
- Protect against layer 7 DDoS attacks and web vulnerabilities using Cloud Armor WAF rules.
- Centralize credential and secret management securely via Secret Manager.
- Apply the principle of least privilege across all GCP IAM roles and service accounts.

2.8 Scalability and future enhancements

Scalability

- Deploy GKE Horizontal Pod Autoscalers linked to CPU and request metrics to handle 100,000 concurrent users.
- Utilize Cloud CDN and global load balancing to offload static traffic and optimize global user latency.
- Implement Memorystore Redis clustering to prevent database bottlenecks under heavy read loads.

Future enhancements

- Incorporate automated canary deployments and progressive delivery pipelines via CI/CD.
- Expand multi-region active-active database configurations using Google Cloud Spanner for zero RPO.
- Integrate advanced AI-driven anomaly detection for real-time security monitoring.

3. Solution design

Recommended architecture

Style: Distributed cloud-native microservices with global CDN

User requests hit a Global HTTP(S) Load Balancer protected by Cloud Armor and cached via Cloud CDN. Traffic is routed to auto-scaling GKE containers across multiple regions running Next.js and Node.js backends. Data is persisted in multi-region Cloud SQL / Spanner databases and accelerated using Redis Memorystore, with Firebase Hosting serving the frontend static assets.

- Edge & Security Layer: Cloud Load Balancing, Cloud CDN, Cloud Armor
- Compute Layer: GKE Multi-Region Clusters with Horizontal Pod Autoscaling
- Application Layer: Next.js SSR Frontend and Node.js REST Backend Services
- Data & Caching Layer: Cloud SQL / Spanner Multi-Region and Memorystore Redis
- Ops & Security Layer: Secret Manager, Cloud KMS, IAM, Terraform IaC

Agents used

- **frontend:** needed for fullstack
- **backend:** needed for fullstack
- **database:** needed for fullstack
- **cloud:** needed for cloud, hosting/infrastructure is in scope
- **security:** needed for fullstack, needed for cloud
- **performance:** needed for fullstack, needed for cloud

Components

Component	Responsibility	From
App Router Shell	Manages core layouts, navigation, and top-level routing for the web application.	frontend
Dashboard Module	Provides high-throughput user interfaces and telemetry views for end-users and administrators.	frontend
Data Synchronization Layer	Handles client-side caching, optimistic updates, and background data synchronization.	frontend
API Gateway and Load Balancer	Distribute incoming global traffic across multi-region backend services with SSL termination and DDoS protection.	backend
Backend Compute Service	Execute core business logic, handle API requests, and auto-scale based on traffic demands.	backend
Database Layer	Persist relational data with cross-region replication to guarantee low RPO and RTO.	backend
Caching and Session Store	Store user session data and cache frequent queries for low latency and high throughput.	backend
Primary Database	Store transactional relational data with strong consistency and multi-region replication.	database
Caching Layer	Provide low-latency session management and frequently accessed data caching.	database
Object Storage	Store user-uploaded media and static assets with high durability.	database
Global Load Balancer	Distribute incoming web traffic globally across multi-region server backends and provide DDoS protection via Cloud Armor.	cloud
Containerised Compute Cluster	Run auto-scaling backend services and frontend containers securely across multiple zones and regions.	cloud
Distributed Relational Database	Provide horizontally scalable, strongly consistent transactional data storage with cross-region replication for disaster recovery.	cloud
Object Storage and CDN	Store static assets and cache them at edge locations to minimize user latency and offload application compute.	cloud
Identity and Access Management	Enforce principle of least privilege using Google Cloud IAM.	security
Secrets Management	Securely store and manage database credentials, API keys, and certificates using Secret Manager.	security
Data Encryption	Ensure encryption at rest and in transit using Cloud KMS and native GCP encryption.	security
Edge Security	Protect the application from DDoS and layer 7 attacks using Cloud Armor.	security
Global Load Balancing and Edge Caching	Distributes incoming traffic across multi-region backends and caches static assets globally to minimize latency.	performance
Auto-scaling Compute Infrastructure	Dynamically scales backend instances based on traffic load to handle peaks of 100,000 users without performance degradation.	performance
In-Memory Caching Layer	Reduces database read load and accelerates data access times using managed in-memory stores.	performance

Recommended technology stack

Category	Choice	Rationale	Alternatives	From
frontend_framework	Next.js	Provides built-in Server-Side Rendering (SSR) and Static Generation (SSG) for optimal global performance and SEO.	React, Vue	frontend
styling_ui	Tailwind CSS	Enables rapid, consistent styling with minimal bundle overhead and highly responsive utilities.	Material UI, CSS Modules	frontend
state_data_fetching	TanStack Query	Robust server-state management handling caching, deduplication, and background refetching out of the box.	Redux Toolkit, Zustand	frontend
backend_runtime	Node.js	Provides excellent asynchronous I/O performance and horizontal scalability ideal for handling 100,000 concurrent web users.	Go, Python	backend
api_style	REST	Standard, robust protocol for frontend-to-backend communication supported natively by GCP load balancers and API gateways.	GraphQL, gRPC	backend
database	Cloud SQL	Managed relational database service offering built-in high availability, automated backups, and cross-region replicas for disaster recovery.	AlloyDB	backend, database
database	Memorystore	Fully managed in-memory data store service for Redis to handle rapid session management and caching.	Redis	backend, database, performance
database	Spanner	Offers globally distributed, horizontally scalable relational database capabilities with high availability and low RTO/RPO for disaster recovery.	AlloyDB	cloud
auth	Identity Platform	Enterprise-grade identity and access management service natively integrated into GCP to handle secure authentication.	Auth0, Keycloak	backend
gcp_compute_network	GKE	Provides powerful container orchestration and automated horizontal pod autoscaling to handle fluctuating loads up to 100,000 users.	Cloud Run, Compute Engine	backend, cloud, performance
gcp_compute_network	Cloud Load Balancing	Global external HTTP(S) load balancing distributes user traffic efficiently to the nearest healthy backend region with failover.	Compute Engine Load Balancing, Compute Engine TCP/UDP Load Balancing, Compute Engine	backend, cloud, performance
gcp_compute_network	Cloud CDN	Caches static assets at edge locations globally to reduce latency and origin server load.	Cloud Storage, Third-party CDN	backend, cloud, performance
gcp_compute_network	Cloud Armor	Protects the application against DDoS attacks and web application vulnerabilities.	Cloud IDS, Cloud Firewall rules, Third-party WAF	backend, cloud, security
gcp_data_messaging	Cloud Storage	Provides multi-region object storage with 99.999999999% durability for user files and media backups.	Local disk, Persistent Disk	database, cloud

Category	Choice	Rationale	Alternatives	From
gcp_ops_security	Secret Manager	Securely stores and manages application credentials, API keys, and database passwords.	Environment variables in Kubernetes secrets, Environment Variables, HashiCorp Vault	backend, cloud, security
gcp_ops_security	IAM	Provides fine-grained access control to GCP resources following the principle of least privilege.	Basic Roles	security
gcp_ops_security	Cloud KMS	Enables customer-managed encryption keys (CMEK) for rigorous data protection at rest.	Google-managed keys	security
hosting_static	Firebase Hosting	Delivers fast static and dynamic edge-hosted content globally via a resilient CDN infrastructure.	Vercel, Netlify	frontend
devops	Terraform	Enables Infrastructure as Code (IaC) to spin up identical secondary disaster recovery regions reliably.	Kubernetes, Deployment Manager	backend, cloud
testing_quality	Playwright	Ensures end-to-end reliability and cross-browser functional testing for complex user workflows.	Cypress, Jest	frontend

4. Resources and estimate

Required human resources

Role	FT E	Seniority	Responsibilities	Phases
Cloud Architect	1	senior	Design and oversee multi-region GCP infrastructure, Terraform IaC, and disaster recovery strategy.	Discovery & Architecture, Infrastructure Setup, Testing & Hardening, Launch
Backend Developer	2	senior	Implement Node.js REST services, database integration, caching mechanisms, and GKE configurations.	Infrastructure Setup, Backend Build, Testing & Hardening, Launch
Frontend Developer	2	mid	Build Next.js App Router, Tailwind UI components, TanStack Query integration, and Playwright E2E tests.	Backend Build, Testing & Hardening, Launch
DevOps Engineer	1	senior	Automate CI/CD pipelines, configure GKE clusters, set up multi-region networking and Terraform modules.	Infrastructure Setup, Backend Build, Testing & Hardening, Launch
QA Engineer	1	mid	Conduct automated E2E testing, security vulnerability scans, and high-load performance testing.	Testing & Hardening, Launch
Project Manager	0.5	mid	Manage sprint planning, stakeholder communications, and delivery tracking.	Discovery & Architecture, Infrastructure Setup, Backend Build, Testing & Hardening, Launch

Team size: 7.5 FTE across 6 roles.

Required AI and technical resources

Type	Resource	Purpose	Sizing
cloud compute	Google Kubernetes Engine (GKE)	Container orchestration and multi-region application scaling	Multi-region cluster with auto-scaling node pools (min 3 nodes per region)
cloud compute	Cloud Load Balancing	Global external HTTP(S) load balancing and traffic routing	Global anycast IP with multi-region backend failover
cloud compute	Cloud Armor	DDoS protection and Web Application Firewall (WAF)	Enterprise security policies applied to load balancer
cloud storage	Cloud CDN	Global edge caching for static assets and reduced origin load	Global edge points of presence
cloud storage	Cloud Storage	Object storage for media files and backups	Multi-region bucket with 99.999999999% durability
database	Cloud SQL	Primary relational database with cross-region replication	Enterprise tier with synchronous cross-region standby replicas
database	Memorystore for Redis	In-memory caching and session management	Standard tier Redis cluster mode enabled
dev tooling	Terraform	Infrastructure as Code for multi-region environment provisioning	Version-controlled infrastructure repository
hosting	Firebase Hosting	Hosting and delivering static Next.js/frontend assets	Global CDN hosting tier
monitoring	Google Cloud Operations Suite	Logging, monitoring, and tracing across multi-region services	Standard log retention and custom metric alerts
testing	Playwright	End-to-end and cross-browser functional testing	CI pipeline execution suite
third party service	Identity Platform	Enterprise user authentication and session management	Managed GCP authentication service tier

No AI models or services are needed for this solution.

Estimated development effort

Phase	Roles	Low	Likely	High
Discovery & Architecture	Cloud Architect, Project Manager	10	15	20
Infrastructure Setup	Cloud Architect, DevOps Engineer, Project Manager	20	25	35
Backend & Frontend Build	Backend Developer, Frontend Developer, DevOps Engineer, Project Manager	40	55	70
Testing & Hardening	Cloud Architect, Backend Developer, Frontend Developer, DevOps Engineer, QA Engineer, Project Manager	20	30	40
Launch & Handover	Cloud Architect, DevOps Engineer, Project Manager	5	10	15
Total (person-days)		95	135	180

About **6.8 person-months** likely (20 working days per month).

Estimated timeline

Phase	Weeks	Duration	Depends on	Schedule
Discovery & Architecture	1–3	3 wk	-	
Infrastructure Setup	3–6	4 wk	Discovery & Architecture	
Backend & Frontend Build	5–10	6 wk	Infrastructure Setup	
Testing & Hardening	9–12	4 wk	Backend & Frontend Build	
Launch & Handover	13–14	2 wk	Testing & Hardening	

Total: about 14 weeks with the team above (range 9.9–18.7 weeks, following the effort range).

What could change the estimate:

- Unforeseen cross-region database replication lag during peak load testing.
- Complexities in multi-region GKE ingress routing and global SSL certificate provisioning.
- Delays in third-party identity provider configurations or security audit approvals.

5. Specialist findings

frontend

Frontend architecture recommending Next.js for high-performance server-side rendering, Tailwind CSS for consistent UI styling, and robust accessibility standards to support 100,000 active users.

- Adhere to WCAG 2.2 AA standards across all interactive components, ensuring high contrast, correct semantic markup, and full keyboard navigation.
- Implement code splitting and lazy loading per route to optimize initial page load times for global users.

backend

Backend architecture design for a highly available, multi-region GCP web application supporting 100,000 users with zero-downtime failover and automated disaster recovery.

- Deploy GKE clusters across two distinct Google Cloud regions (active-passive or active-active) to fulfill 99.99% availability requirements.
- Configure Cloud SQL with synchronous replication to a standby replica in a secondary region to achieve minimal RPO and RTO.
- Implement Horizontal Pod Autoscaler (HPA) within GKE linked to custom metrics to handle traffic spikes smoothly.

database

Database and data storage architecture designed for high availability, multi-region replication, and disaster recovery to support 100,000 active users on GCP.

- Configure Cloud SQL with a high availability setup across two zones within the primary region, and establish a cross-region read/failover replica for disaster recovery.
- Utilize Memorystore with Redis cluster mode enabled for session caching and low latency responses.

- Implement automated point-in-time recovery and continuous backup policies to meet low RPO targets.

cloud

Cloud architecture design for a highly available GCP web application supporting 100,000 users with multi-region failover and disaster recovery.

- Deploy GKE clusters in an active-active configuration across at least two GCP regions to meet the 99.99% availability target.
- Utilize Google Cloud Spanner multi-region instances to ensure near-zero RPO and low RTO during a regional failover event.
- Implement Horizontal Pod Autoscaling (HPA) inside GKE tied to CPU and request metrics to dynamically scale for 100,000 concurrent users.
- Configure Cloud CDN in front of Cloud Storage and load balancers to minimize latency for global users.

security

Security, identity, access control, and data protection strategy for the High Availability GCP Web Application.

- Enforce HTTPS for all in-transit traffic using TLS 1.3.
- Restrict internal microservice communication using VPC service controls and firewall rules.
- Audit all administrative actions using Cloud Audit Logs.

performance

Performance, scaling, caching, and bottleneck reduction strategy for supporting 100,000 concurrent users with 99.99% availability on Google Cloud Platform.

- Implement multi-region active-active or active-passive architecture to guarantee a low RTO and RPO during regional outages.
- Utilize HTTP/3 and edge caching policies to optimize asset delivery and Core Web Vitals.
- Configure Horizontal Pod Autoscaling (HPA) in GKE driven by CPU and custom request-rate metrics.

6. Alternatives

Listed per technology choice in section 3.

7. Risks

Risk	Severity	Likelihood	Mitigation	From
Regional GCP outage affecting the primary deployment zone.	critical	low	Automated global external HTTP(S) load balancing health checks to route traffic to the secondary disaster recovery region.	backend
Exposure of sensitive database credentials in application code or configuration.	critical	low	Store all secrets in Secret Manager and fetch them at runtime securely.	security
Regional infrastructure failure disrupts application availability.	critical	low	Configure Cloud Load Balancing with multi-region backend failover and automated health checks.	performance
Database replication lag during peak traffic spikes.	high	medium	Provision adequate network bandwidth between regions and scale Cloud SQL instance tiers appropriately.	backend
Cross-region database replication lag during peak traffic spikes.	high	medium	Use Google Cloud Spanner multi-region configurations to manage consistency models and minimize replication delays automatically.	cloud
Unauthorized access due to overly permissive IAM roles.	high	medium	Apply the principle of least privilege and regularly audit IAM policies using IAM Recommender.	security
Layer 7 DDoS attacks overwhelming the application tier.	high	medium	Deploy Cloud Armor security policies with rate limiting and pre-configured OWASP Top 10 rules.	security
Database becomes a bottleneck under sudden spikes of 100,000 users.	high	medium	Deploy Memorystore for aggressive query caching and implement read replicas across multi-zones.	performance
High latency for global users accessing static assets from a single region.	medium	medium	Deploy frontend assets to a global CDN edge caching layer.	frontend
Replication lag during peak load causing stale reads.	medium	medium	Route critical writes and reads requiring strong consistency to the primary instance, while routing read-only queries to replicas.	database
Sudden traffic surges exceeding the provisioned autoscaling limits.	medium	medium	Pre-warm GKE node pools or set appropriate minimum node counts to absorb sudden spikes while cluster autoscaler reacts.	cloud

8. Quality score

Overall: 92.0/100

Dimension	Weight %	Score
requirement_fit	22.5	95
architecture	17.5	90
security	17.5	95
reliability	15.0	95
performance	12.5	90
cost	7.5	80
maintainability	7.5	90

9. Assumptions

- Standard web application architecture with a database and backend services
- Multi-region or multi-zone GCP deployment is required for high availability and disaster recovery
- Static assets can be hosted externally on a global content delivery network while API calls route back to the GCP backend infrastructure.
- Traffic patterns can be managed using standard horizontal scaling.
- Data compliance rules allow cross-region replication for disaster recovery purposes.
- Relational data model is sufficient for the application domain.
- Multi-region replication handles regional failover scenarios adequately.
- Application is containerized and stateless, allowing easy distribution across GKE pods.
- Database schema is compatible with distributed SQL semantics provided by Spanner or Cloud SQL enterprise configurations.
- All data in transit must be encrypted using TLS.
- Compliance standards like SOC2 or GDPR may apply depending on the user base.
- Application backend is containerized for deployment on GKE.
- Database layer supports replication for multi-zone and multi-region fault tolerance.
- Client provides necessary GCP cloud billing accounts and organization permissions.
- Application source code requirements and wireframes are established during discovery.
- Third-party compliance certifications (SOC2/GDPR) guidelines are provided prior to build.

10. Review history

Round	Decision	Score	Findings (severity)
1	APPROVED	92.0	none